

Backend and Databases

A beginner-friendly book using FixHub as the running example

Purpose of this guide

This PDF explains backend and database ideas from the ground up. It assumes you have never studied backend development before. FixHub is used as the example application so every concept connects to something real: repairs, customers, inventory, invoices, team members, branches, and customer repair tracking.

Version: FixHub V1 learning edition

Best way to read: one chapter per sitting. Read slowly. After each chapter, answer the practice questions before moving on.

Table of Contents

1. The Big Picture: What Is a Backend?
 2. Websites, Frontends, Backends, and Databases
 3. The Journey of One FixHub Request
 4. What an API Is and Why It Matters
 5. Databases: The Memory of an Application
 6. Tables, Rows, Columns, and Relationships
 7. SQL and PostgreSQL Basics
 8. How FixHub's Database Is Organized
 9. Prisma: The Translator Between Code and Database
 10. Migrations: Changing the Database Safely
 11. Authentication and Authorization
 12. Admin Users vs Customer Users in FixHub V1
 13. Deployment, Environment Variables, and Production
 14. Debugging Backend and Database Errors
 15. V1 Launch Checklist and Practice Exercises
 16. How to Read the FixHub Backend Files
 17. How to Design a Database From Real Business Work
 18. A 30-Day Learning Plan
- Glossary

1. The Big Picture: What Is a Backend?

When people open FixHub in their browser, they see buttons, forms, tables, and pages. That visible part is the frontend. The frontend is important, but it cannot do everything by itself. It needs a place to save information, check passwords, decide permissions, create invoices, and return repair records. That hidden working part is the backend.

A backend is the part of an application that runs on a server. A server is simply a computer that is always available to receive requests. In FixHub, the backend receives requests like: login this staff member, create this repair ticket, list all inventory items, create this invoice, or track this repair for a customer.

Simple definition

The backend is the private engine of the application. It receives requests, applies business rules, talks to the database, and sends back a response.

Think of FixHub like a repair shop. The frontend is the reception desk the user can see. The backend is the workshop behind the door. The database is the filing cabinet and accounting ledger. A customer may only see the tracking counter, but staff can enter the workshop and use more tools.

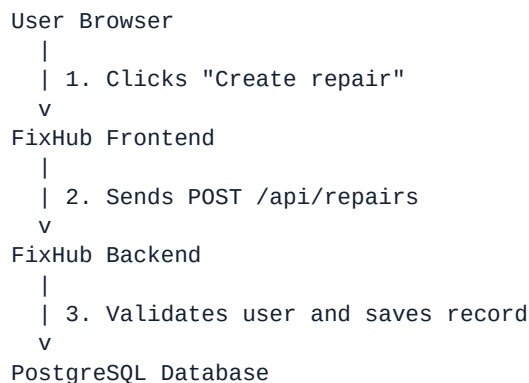
Part	In FixHub	Main job
Frontend	fixhub-client	Shows pages and forms in the browser.
Backend	fixhub-server	Receives requests, checks rules, and sends responses.
Database	PostgreSQL/Neon or local Postgres	Stores users, repairs, customers, inventory, invoices, and branches.
API	Express routes like /api/repairs	A controlled doorway between frontend and backend.

If the frontend is beautiful but the backend is broken, users will see errors. If the backend is good but the database is missing a table, the backend may crash. If the database is fine but the frontend sends the wrong data, the request may fail. Real applications work because all three layers cooperate.

2. Websites, Frontends, Backends, and Databases

A website can be simple or complex. A simple website may only show static text and images. A business application like FixHub is different. It has accounts, permissions, records, status changes, payments, and private data. That means it needs both a backend and a database.

The frontend lives in the user's browser. It asks the backend for information. The backend runs on a server. It asks the database for stored information. The database stores the long-term truth.



Notice the direction. The browser does not connect directly to the database. That would be unsafe. The backend sits in the middle so it can check identity, enforce permissions, validate data, and hide database secrets.

Important mental model

The frontend asks. The backend decides. The database remembers.

For FixHub V1, admin users use the full dashboard. Customer users should not see that dashboard. Customers should only use the public tracking page. That separation is a backend and frontend design decision, not just a visual design choice.

3. The Journey of One FixHub Request

To understand backend and databases, follow one action from beginning to end. Imagine a Front Desk staff member creates a new repair ticket for a phone.

Step	What happens	Where
1	Staff fills the New Repair form.	Frontend
2	Frontend sends data to /api/repairs.	API request
3	Backend checks the staff login token.	Auth middleware
4	Backend validates fields like customer name and device.	Service layer
5	Backend creates a Repair row in the database.	PostgreSQL
6	Backend sends the saved repair back to the frontend.	JSON response
7	Frontend updates the screen so staff sees the new ticket.	Frontend

A request is like a short conversation. The frontend says: please do this. The backend says: I will check if this is allowed. The database says: here is the record, or I saved it. Then the backend replies to the frontend.

```
POST /api/repairs
Authorization: Bearer <staff-token>
Content-Type: application/json

{
  "customer": "Sam Wright",
  "device": "iPhone 12",
  "issue": "Screen replacement",
  "status": "Received"
}
```

The response is usually JSON. JSON is a text format that both frontend JavaScript and backend JavaScript understand easily.

```
{
  "id": 8,
  "ticketNumber": "FH-20260718-A1B2C3",
  "customer": "Sam Wright",
  "device": "iPhone 12",
  "status": "Received"
}
```

4. What an API Is and Why It Matters

API means Application Programming Interface. That sounds heavy, but the idea is simple: an API is a set of agreed doors into an application. Each door has a path, a method, rules, and a response.

In FixHub, the backend API has paths like `/api/auth/login`, `/api/repairs`, `/api/inventory`, `/api/invoices`, `/api/team`, `/api/branches`, and `/api/tracking`.

HTTP method	Meaning	FixHub example
GET	Read something	GET <code>/api/repairs</code> lists repair tickets.
POST	Create or perform an action	POST <code>/api/invoices</code> creates an invoice.
PUT	Replace or update something	PUT <code>/api/repairs/8</code> updates repair 8.
DELETE	Remove something	DELETE <code>/api/customers/3</code> deletes customer 3.

APIs help keep the application organized. The frontend does not need to know database commands. It only needs to know which API path to call. The backend does not need to know how the page looks. It only needs to receive data and return data.

Why APIs matter

A good API is like a menu in a restaurant. Customers do not walk into the kitchen and cook. They choose from the menu. The backend API is the menu of safe actions the frontend can request.

FixHub also has public and private API doors. Most doors require a logged-in admin or staff user. The tracking door is public, but it still checks ticket number plus contact detail so random people cannot view repair information.

5. Databases: The Memory of an Application

A database is where an application stores information permanently. If FixHub only stored data in the browser, the data could disappear when the browser is closed or when another staff member logs in. A database gives the application shared memory.

FixHub stores business records such as users, customers, repair tickets, inventory items, invoices, payments, branches, notification logs, stock movements, and audit logs.

Without a database	With a database
Repair tickets disappear when the app restarts.	Repair tickets stay available.
Only one browser may know the data.	All authorized staff can see shared records.
No reliable invoice history.	Invoices and payments can be tracked.
No long-term reporting.	Dashboard and insights can be calculated.

The database is not just a storage box. It has structure. It can enforce unique values, relationships, and indexes. For example, an invoice can be connected to a repair ticket. A repair can belong to a branch. An inventory movement can belong to an inventory item.

Truth source

In a serious application, the database is the source of truth. The frontend displays truth, and the backend protects truth, but the database stores truth.

6. Tables, Rows, Columns, and Relationships

Most business applications use relational databases. PostgreSQL is a relational database. Relational means data is organized into tables that can connect to each other.

Database word	Plain meaning	FixHub example
Table	A category of records	Repair table
Row	One record inside a table	Repair #8
Column	One field on each row	status, customer, device
Primary key	Unique id for a row	Repair.id
Foreign key	Connection to another table	Repair.userId points to User.id

Imagine a spreadsheet named Repair. Each row is one repair job. Each column is a piece of information about that job. A database table is more powerful than a spreadsheet because it can enforce rules and relationships.

Repair table

```
id | ticketNumber      | customer  | device   | status
---+-----+-----+-----+-----
8  | FH-20260718-A1B2C3 | Sam Wright | iPhone 12 | Diagnosing
9  | FH-20260718-X9Y8Z7 | Ada Okoro  | HP Laptop | Ready
```

Relationships are what make databases powerful. A Customer can have many Repairs. A Repair can have one Invoice. An Invoice can have many Payments. A Branch can have many Repairs and Inventory items.

User

```
|
+-- many Repairs
+-- many Customers
+-- many Inventory items
+-- many Invoices
```

Repair

```
|
+-- one Invoice
+-- many StatusHistory records
```

7. SQL and PostgreSQL Basics

SQL means Structured Query Language. It is the language used to speak directly to many relational databases, including PostgreSQL. You do not need to memorize SQL immediately, but you should understand the basic shapes.

When FixHub uses Prisma, Prisma generates SQL behind the scenes. But the database still understands SQL.

```
-- Read repairs
SELECT * FROM "Repair";

-- Read one repair by id
SELECT * FROM "Repair" WHERE "id" = 8;

-- Create a branch
INSERT INTO "Branch" ("name", "userId")
VALUES ('Main Branch', 1);

-- Update repair status
UPDATE "Repair"
SET "status" = 'Ready'
WHERE "id" = 8;
```

SQL action	What it does
SELECT	Read rows from a table.
INSERT	Create new rows.
UPDATE	Change existing rows.
DELETE	Remove rows.
JOIN	Read connected data from multiple tables.

Careful with SQL

SQL is powerful. A wrong DELETE or UPDATE can damage data. This is one reason apps use service layers, migrations, backups, and careful production workflows.

8. How FixHub's Database Is Organized

FixHub's database is organized around the daily work of a repair business. The main tables model real things in the shop.

Table	Stores	Example
User	Admin and staff accounts	Owner, Admin, Technician, Front Desk
Customer	People who bring devices for repair	Name, phone, email
Repair	Repair jobs and statuses	Device, issue, status, due date
RepairStatusHistory	Timeline of status changes	Received, Diagnosing, Ready
Inventory	Parts and stock	iPhone screen, battery
StockMovement	Inventory changes	Added 10 screens, used 1 screen
Invoice	Bills connected to repairs	Total, amount paid, balance
InvoiceItem	Line items on invoices	Screen replacement part
Payment	Payments received	Cash, transfer, card
Branch	Shop locations or service centers	Main Branch, Island Branch
NotificationLog	Messages prepared or sent	SMS, WhatsApp, Email
AuditLog	Important actions	User created invoice

The schema is like the blueprint of the database. It describes which tables exist, which columns they have, and how tables are connected.

```
model Repair {
  id          Int          @id @default(autoincrement())
  customer    String
  device      String
  status      String
  userId      Int
  branchId    Int?
  invoice     Invoice?
}
```

The question mark after a type means optional. For example, `branchId` can be null because a repair might exist before branch support is fully used. Optional fields are useful, but too many optional fields can make the app harder to reason about.

9. Prisma: The Translator Between Code and Database

Prisma is an ORM. ORM means Object-Relational Mapper. In plain language, Prisma lets JavaScript code talk to the database using JavaScript-like commands instead of writing raw SQL every time.

In FixHub backend code, you may see commands like `prisma.repair.findMany()`, `prisma.user.findUnique()`, or `prisma.invoice.create()`. Prisma translates these into SQL and sends them to PostgreSQL.

Prisma code	Plain meaning
<code>prisma.repair.findMany()</code>	Find many repair rows.
<code>prisma.user.findUnique({ where: { email } })</code>	Find one user by unique email.
<code>prisma.inventory.create({ data })</code>	Create an inventory row.
<code>prisma.invoice.update({ where, data })</code>	Update an invoice row.

```
const repair = await prisma.repair.findUnique({
  where: {
    id: 8,
  },
});
```

Prisma is helpful because it understands your schema. If the schema says Repair has a status field, Prisma knows about `repair.status`. But this also means Prisma can fail if the application schema expects a table or column that the production database does not actually have.

The error you saw

When FixHub showed errors like 'The column User.role does not exist', Prisma was saying: the code expects this column, but the current database does not have it. That is a schema mismatch.

10. Migrations: Changing the Database Safely

Applications grow. At first FixHub may only need users and repairs. Later it needs invoices. Later it needs branches and team permissions. Every time the database structure changes, the database needs a controlled update. That controlled update is called a migration.

A migration is a file that says: add this table, add this column, create this index, or connect these tables. Prisma migrations live in the `prisma/migrations` folder.

Migration example	What it means
20260624090000_repair_workflow	Added repair workflow fields.
20260625083640_add_invoice_model	Added invoice tables.
20260702090000_team_permissions	Added User.role and workspace owner support.
20260709090000_multi_shop_branches	Added branches.
20260712195634_uploads_and_audit	Added uploads/audit features.

```
npx prisma migrate deploy
```

That command applies pending migrations to a database. In production, you must run migrations against the production database. Running migrations only on a local database does not update Neon or Vercel production.

Why migrations matter

Most of the production errors you saw were migration problems. The code had moved forward, but the production database had not fully caught up.

Runtime fallbacks can help rescue users from crashes, but they should not replace proper migrations forever. For a clean launch, the real database should match the schema.

11. Authentication and Authorization

Authentication means proving who someone is. Authorization means deciding what that person is allowed to do.

Concept	Question it answers	FixHub example
Authentication	Who are you?	Login with email and password.
Authorization	What can you do?	Owner can manage team; Technician can work on repairs.

FixHub uses login credentials to create a token. The frontend stores that token and sends it with private API requests. The backend reads the token, verifies it, finds the user, and attaches the user information to the request.

```
Authorization: Bearer <token>
```

The token is not a password. It is more like a temporary signed pass. If the token is valid, the backend can trust that the request came from that user. If the token is missing or invalid, private routes return 401 Unauthorized.

Roles are authorization tools. FixHub roles include Owner, Admin, Manager, Technician, Front Desk, and Receptionist. In V1, these are admin/staff users, not customer accounts.

Do not confuse users and customers

A User is someone who logs into the FixHub admin system. A Customer is someone whose device is being repaired. In V1, customers track repairs publicly with ticket and contact verification.

12. Admin Users vs Customer Users in FixHub V1

This is a major product decision for V1. The fastest way to confuse new users is to give every person a dashboard before they understand the core workflow. FixHub V1 should keep the user types simple.

Person	Needs account?	Access
Owner	Yes	Everything, including team, branches, invoices, settings-like actions.
Admin/Manager	Yes	Daily operations and staff management, depending on permissions.
Technician	Yes	Repair work, inventory context, status updates.
Front Desk/Receptionist	Yes	Customer intake, repair creation, invoices, communication.
Customer	No for V1	Public tracking page only.

For customers, the V1 question is not: what dashboard do they need? The better question is: what one thing are they trying to know? Usually: what is happening with my repair?

That is why the customer tracking page is enough for V1. It lets customers enter ticket number plus phone/email and see progress. This gives value without forcing account creation.

V1 recommendation

Launch customers with tracking only. Add customer accounts in V2 only if there is a clear reason: repeat customers, warranty history, quotes approval, payments, or messaging.

The admin dashboard is the shop control room. The tracking page is the customer window. Keep those separate.

13. Deployment, Environment Variables, and Production

Deployment means putting your application on the internet so real users can use it. FixHub has a client deployment and a server deployment. The client is the frontend website. The server is the backend API.

Thing	Example
Frontend URL	https://fixhub-client.vercel.app
Backend URL	https://fixhub-server.vercel.app
Database	PostgreSQL, often hosted by Neon
Production	The live version users actually use
Local development	The copy running on your computer

Environment variables are settings that should not be hard-coded into source code. They include `DATABASE_URL`, `JWT_SECRET`, and `CLIENT_URL`. `DATABASE_URL` tells the backend where the database is. `JWT_SECRET` signs login tokens. `CLIENT_URL` tells the backend which frontend URL is allowed.

Password safety

Never paste real database passwords, JWT secrets, API keys, or full production URLs into public chat, GitHub commits, screenshots, or documents. If a secret is exposed, rotate it.

```
DATABASE_URL=postgresql://user:password@host/database
JWT_SECRET=a-long-random-secret
CLIENT_URL=https://fixhub-client.vercel.app
```

In production, these values are usually set in the hosting provider dashboard, such as Vercel. The code reads them at runtime. That way, the same code can run locally and in production with different settings.

14. Debugging Backend and Database Errors

Backend/database errors can look frightening, but they usually tell you exactly where to look. The trick is to read them slowly.

Error clue	Likely meaning	FixHub example
table does not exist	Migration missing or wrong database	public.Branch missing
column does not exist	Migration missing or schema mismatch	User.role missing
Authentication failed	Wrong database password or URL	Neon password issue
Cannot reach database	Network, host, or direct/pooler issue	Direct Neon host unavailable
401 Unauthorized	Missing or invalid login token	Admin API called while logged out
403 Forbidden	Logged in but role not allowed	Technician tries owner-only action

A good debugging process is not panic. It is a checklist.

- Read the exact error message.
- Identify whether it is frontend, backend, database, or deployment.
- Check whether the failing route is public or protected.
- Check whether local database and production database are the same.
- Check whether migrations have been applied to the database receiving requests.
- Check whether environment variables are present in the deployment.
- Fix the smallest cause, then test the exact failing flow again.

For example, 'Invalid prisma.user.findMany() invocation: The column User.role does not exist' means Prisma tried to read User.role. The database said no such column exists. That is not a CSS problem or browser problem. It is a database schema problem.

Slow is smooth

When debugging, do not change five things at once. Change one thing, test one thing, and keep notes. Production debugging rewards calm sequence.

15. V1 Launch Checklist and Practice Exercises

FixHub V1 should launch with a tight, understandable workflow. Do not launch every possible feature idea at once. Launch the repair-shop control room and public tracking window.

Area	V1 expectation
Landing page	Explains FixHub and leads to login/tracking.
Admin login	Staff can sign in securely.
Dashboard	Shows operational overview without errors.
Repairs	Create, list, view, and update repair tickets.
Customers	Create and manage customer records.
Inventory	Add parts and track stock.
Invoices	Create invoices and payments.
Branches	Manage locations.
Team	Manage staff accounts and roles.
Customer tracking	Public ticket lookup with contact verification.

Practice exercises:

- Explain the difference between frontend, backend, and database in your own words.
- Draw the journey of a user logging into FixHub.
- Name five FixHub database tables and what each stores.
- Explain why customers should not access the admin dashboard in V1.
- Explain what a migration does and why missing migrations caused FixHub errors.
- Write a simple sentence explaining what an API route is.
- Look at a Prisma error and decide if it is about a table, a column, auth, or network.

Your learning path

Do not try to learn everything in one day. Learn request flow first. Then database tables. Then relationships. Then migrations. Then auth. Those five ideas unlock most backend understanding.

16. How to Read the FixHub Backend Files

A backend project can look intimidating because it has many folders. The trick is to know what each folder is responsible for. Do not read the whole project randomly. Follow the request path.

Folder or file	What it usually means in FixHub
src/server.js	Starts the Express app, configures middleware, and connects routes.
src/routes	Defines API doors such as /api/repairs and /api/auth/login.
src/controllers	Receives a route request and prepares the HTTP response.
src/services	Contains business logic like creating invoices or resolving branches.
src/middleware	Runs before controllers for auth, permissions, logging, rate limits, and errors.
src/lib/prisma.js	Creates the Prisma client used to talk to the database.
prisma/schema.prisma	The database blueprint Prisma uses.
prisma/migrations	Historical database structure changes.

If you want to understand one feature, start at the route. For example, to understand Team, read teamRoutes.js first. It tells you which controller functions are used and which permissions protect them. Then open teamController.js. Then open teamService.js. Then check schema.prisma to see what database table the service is using.

Request path for Team:

```
src/server.js
-> app.use("/api/team", teamRoutes)

src/routes/teamRoutes.js
-> router.get("/", getTeam)
-> router.post("/", managers, createMember)

src/controllers/teamController.js
-> calls teamService.listTeam()
-> calls teamService.createTeamMember()

src/services/teamService.js
-> uses prisma.user.findMany()
-> uses prisma.user.create()

prisma/schema.prisma
-> model User
```

Reading habit

When learning a backend, follow one request from route to controller to service to database. That path is the spine of the application.

This method also helps with debugging. If the frontend says Team failed, you can follow the team request path. If the error mentions `prisma.user.findMany()`, you know the failure is in a service call to the User table, not in the button design.

17. How to Design a Database From Real Business Work

Database design starts with the real business, not with code. For FixHub, the business is a repair operation. A customer brings a device. Staff creates a repair. The repair moves through statuses. Parts may be used. An invoice may be created. Payment may be collected. A customer may track progress.

To design a database, identify nouns and verbs. Nouns often become tables. Verbs often become actions, logs, or status changes.

Business sentence	Noun/table	Verb/action
A customer brings a phone for repair.	Customer, Repair	Create repair
A technician changes the repair status.	RepairStatusHistory	Add status history
A screen is used from inventory.	Inventory, StockMovement	Adjust stock
An invoice is generated for a repair.	Invoice, InvoiceItem	Create invoice
A customer pays part of the invoice.	Payment	Record payment
A shop owner adds another location.	Branch	Create branch

A common beginner mistake is putting everything into one giant table. That may feel simple at first, but it becomes messy quickly. If Repair had every payment, invoice item, stock movement, and notification inside it, the table would be confusing and hard to query.

Good database design

Separate different things into different tables, then connect them with IDs. This keeps data flexible and easier to reason about.

Bad idea:

Repair table has customer fields, invoice fields, payment fields, stock fields, message fields, and team fields all mixed together.

Better idea:

Repair connects to Customer-like information, Invoice, StatusHistory, Branch, and NotificationLog through clear relationships.

Another important question is: what must be historical? Repair status history should be historical because you want to know the timeline, not just the current status. Payments should be historical because a customer can pay in parts. Stock movements should be historical because inventory changes need an audit trail.

18. A 30-Day Learning Plan

Here is a practical 30-day path to understand backend and databases without drowning in jargon. Spend 30 to 60 minutes per day. Use FixHub as the example while studying.

Days	Focus	What to practice
1-3	Frontend vs backend vs database	Draw the three boxes and explain request flow.
4-6	HTTP and APIs	Write examples of GET, POST, PUT, DELETE for FixHub.
7-10	Database tables	List FixHub tables and what each stores.
11-13	Relationships	Draw User -> Repair -> Invoice -> Payment.
14-16	SQL basics	Practice SELECT, INSERT, UPDATE, DELETE on sample tables.
17-19	Prisma	Read simple Prisma calls and translate them into plain English.
20-22	Migrations	Explain why User.role missing caused a production error.
23-25	Authentication	Explain login, token, 401, and protected routes.
26-27	Authorization	Explain Owner, Admin, Technician, Front Desk permissions.
28-29	Deployment	Explain local vs production, env vars, and DATABASE_URL.
30	Review	Explain FixHub architecture to someone else in five minutes.

The goal is not to memorize every command. The goal is to build a mental map. Once you can say where a problem belongs - frontend, backend, database, auth, migration, or deployment - you are no longer lost.

How you know you are improving

You will start reading errors differently. Instead of seeing a scary red message, you will see clues: route, model, table, column, status code, environment, or permission.

At the end of 30 days, come back to the FixHub codebase and trace three flows: login, create repair, and track repair. If you can follow those from page to API to database and back, you understand the heart of backend development.

Glossary

Term	Meaning
API	A controlled way for frontend and backend to communicate.
Backend	Server-side application logic that handles requests and talks to the database.
Database	Persistent storage for application data.
Frontend	The part of the app users see in the browser.
HTTP	The communication protocol used by browsers and APIs.
JSON	A text format for sending structured data.
JWT	A signed token often used after login.
Migration	A controlled change to database structure.
ORM	A tool that lets code interact with a database using programming language objects.
PostgreSQL	A powerful relational database system.
Prisma	The ORM used by FixHub backend.
Primary key	A unique identifier for a row.
Foreign key	A column that links one table to another.
Route	A backend path like <code>/api/repairs</code> .
Schema	The blueprint of database tables, columns, and relationships.
Server	A computer/process that responds to requests.

Final thought: backend and databases are not magic. They are agreements. The frontend agrees how to ask. The backend agrees how to decide. The database agrees how to remember. Once you see those agreements clearly, the whole system becomes less mysterious.